

DISPENSA DELLA LEZIONE

Props e rendering in JSX

Passaggio di dati, rendering condizionale e contenuti flessibili nei componenti React.

CAPITOLO 1

Cosa sono le props

NOTA

Le props sono valori passati da un componente a un altro. Servono per rendere un componente riutilizzabile, perché la struttura resta uguale ma i dati possono cambiare.

In pratica, un componente riceve informazioni dall'esterno e le usa dentro il JSX per mostrare contenuti diversi.

Quando un componente viene usato, si possono aggiungere attributi simili a quelli HTML. In React questi attributi diventano props, cioè dati disponibili dentro il componente. Questo permette di creare componenti generici, utili in più situazioni, senza riscrivere ogni volta lo stesso codice.

Le props vengono raccolte in un oggetto. Se un componente riceve una prop chiamata `title`, il suo valore può essere letto con `props.title`. In questo modo il componente separa la struttura dai dati che deve mostrare.

esempio

```
function Card(props) {  
  return (  
    <div>  
      <h2>{props.title}</h2>  
      <p>{props.text}</p>  
    </div>  
  );  
}  
  
function App() {  
  return (  
    <Card  
      title="Benvenuto"  
      text="Questo contenuto arriva dalle props"  
    />  
  );  
}
```

CAPITOLO 2

Usare le props con map

NOTA

`map()` permette di trasformare ogni elemento di un array in un nuovo elemento JSX. Spesso viene usato per creare una lista di componenti.

Ogni componente generato con `map()` può ricevere props diverse, in base ai dati presenti nell'array.

Quando si lavora con una lista di dati, non si scrive ogni elemento a mano. Si prepara un array e si usa `map()` per ciclare ogni valore. Durante questo passaggio si può creare un componente e passargli le informazioni necessarie tramite props. In questo modo dati e struttura restano ordinati.

Questo approccio è utile perché separa il componente che rappresenta un singolo elemento dalla logica che costruisce tutta la lista. Ogni elemento può ricevere una prop diversa, ad esempio un nome, un prezzo o un titolo.

esempio

```
function Product(props) {  
  return <li>{props.name}</li>;  
}  
  
function App() {  
  const products = ["Pane", "Latte", "Pasta"];  
  
  return (  
    <ul>  
      {products.map((product, index) => (  
        <Product key={index} name={product} />  
      ))}  
    </ul>  
  );  
}
```

CAPITOLO 3

Conditional Rendering in JSX

NOTA

Il conditional rendering permette di mostrare un contenuto oppure un altro in base a una condizione. In React questa scelta viene fatta direttamente dentro il JSX.

Serve quando un'interfaccia non deve essere sempre uguale, ma deve cambiare in base ai dati disponibili o a una situazione precisa.

Renderizzare significa mostrare un contenuto a schermo. Il conditional rendering, quindi, è il rendering condizionale: un elemento viene mostrato solo se una condizione è vera, altrimenti ne viene mostrato un altro oppure non viene mostrato nulla. Questo concetto si collega agli `if` già conosciuti, ma in JSX si applica dentro il codice che costruisce l'interfaccia.

Per i casi semplici si usa spesso l'operatore ternario, che permette di scegliere tra due risultati. È utile quando si vuole cambiare un testo, un messaggio o un blocco di contenuto in modo diretto e leggibile.

esempio

```
function App() {  
  const isLoggedIn = true;  
  
  return (  
    <div>  
      {isLoggedIn ? <p>Accesso effettuato</p> : <p>Accesso non effettuato</p>}  
    </div>  
  );  
}
```

CAPITOLO 4

Short-Circuiting con gli operatori `&&` e `||`

NOTA

Lo short-circuiting è una tecnica basata sugli operatori logici `&&` e `||`, usata per decidere cosa mostrare in JSX senza scrivere un ternario completo.

Con `&&` si mostra un contenuto solo se una condizione è vera. Con `||` si mostra un valore di riserva quando il primo valore è falso, assente o vuoto.

L'operatore `&&` valuta due valori: se il primo è falso, il risultato è quel valore falso e il secondo non viene nemmeno considerato; se il primo è vero, il risultato è il secondo valore. In JSX questo comportamento si usa per mostrare un elemento solo quando una condizione è vera, evitando di scrivere un ternario con un'alternativa vuota.

L'operatore `||` funziona in modo opposto: se il primo valore è vero, il risultato è quel valore e il secondo non viene considerato; se il primo valore è falso, assente o vuoto, il risultato è il secondo valore. In JSX questo è utile per mostrare un testo o un componente predefinito quando manca un dato specifico.

Rispetto all'operatore ternario visto per il conditional rendering, lo short-circuiting è più adatto quando esiste solo un caso da mostrare, oppure quando serve un valore di riserva senza dover scrivere due alternative complete.

esempio

```
function App() {  
  const hasNotification = true;  
  
  return (  
    <div>  
      {hasNotification && <p>Hai una nuova notifica</p>}  
    </div>  
  );  
}
```

esempio

Lo short-circuiting con `&&` si può usare anche per decidere se mostrare o meno un componente, non solo un testo semplice.

```
function Badge(props) {  
  return <span>{props.text}</span>;  
}  
  
function App() {  
  const isPromo = true;  
  
  return (  
    <div>  
      <h2>Prodotto</h2>  
      {isPromo && <Badge text="In promozione" />}  
    </div>  
  );  
}
```

In questo caso il componente `Badge` viene mostrato solo quando `isPromo` è `true`. Se il valore fosse `false`, il componente non verrebbe renderizzato affatto.

CAPITOLO 5

La prop children

NOTA

children è una prop speciale di React. Contiene tutto quello che viene scritto tra il tag di apertura e il tag di chiusura di un componente.

Serve quando un componente deve fare da contenitore e mostrare al suo interno contenuti scelti ogni volta in modo diverso.

Non sempre conviene passare tutto come props separate. In molti casi è più comodo creare un componente che avvolge altri elementi. Quando si scrive contenuto dentro quel componente, React lo inserisce in **props.children**. Questo è utile per box, card, sezioni o blocchi che devono contenere testo o altri elementi JSX.

La differenza principale è questa: una prop normale passa un valore con un nome preciso, mentre **children** raccoglie il contenuto interno del componente. Così il componente può mantenere una struttura fissa ma accettare contenuti flessibili.

esempio

```
function Box(props) {  
  return (  
    <div>  
      {props.children}  
    </div>  
  );  
}  
  
function App() {  
  return (  
    <Box>  
      <h2>Titolo interno</h2>  
      <p>Questo testo passa tramite children.</p>  
    </Box>  
  );  
}
```